

Практикум 5.7. Выбор алгоритма под ограничения

Участники: _____ Дата: _____

Одна задача, десять рабочих решений. Все они дают правильный ответ, пока соблюдены их требования к входу. Выбрать нужно не «самое умное» решение, а то, которое подходит под ограничения заказа.

Порядок работы: вопросы к условию → карта вариантов по файлу → выбор под каждый заказ → проверка запуском → сверка с парой.

Задача А. k-й по величине элемент списка

Условие, как его дают: «Верните k-й по величине элемент списка целых чисел». Больше ничего не сказано. Файл с вариантами: `zadacha_a_kth.py`.

1. Вопросы к условию

Запишите не меньше пяти вопросов, ответы на которые меняют выбор варианта.

1. _____
2. _____
3. _____
4. _____
5. _____

2. Карта вариантов

Заполняется чтением файла. Докстрока каждой функции содержит нужное.

Вариант	Время	Доп. память	Что требует от входа
v01_sort_kopii			
v02_sort_na_meste			
v03_nlargest			
v04_kucha_iz_k			
v05_k_prohodov			
v06_quickselect			

Вариант	Время	Доп. память	Что требует от входа
v07_schetchik			
v08_bisect_top_k			
v09_dve_peremennye			
v10_razlichnye			

3. Заказы

Для каждого заказа выберите один вариант и объясните, почему не соседний.

№	Ограничения заказа	Вариант	Почему он
A1	800 оценок, к любое, список нужен дальше в исходном порядке, память не ограничена		
A2	10 000 000 чисел читаются по одному, нужны 10 наибольших, памяти мало		
A3	Баллы теста — целые от 0 до 100, $n = 1\,000\,000$, ответ нужен быстро		
A4	$n = 200\,000$, дополнительной памяти почти нет, исходный порядок дальше не нужен		
A5	Нужна вторая по величине зарплата, одинаковые зарплаты считаются одной		

4. Варианты, которые не подойдут ни одному заказу

Назовите их и напишите, при каких ограничениях они всё же были бы уместны.

Задача В. Есть ли значение в списке

Условие, как его дают: «Отвечайте на запросы, есть ли число x в списке». Файл с вариантами: `zadacha_b_poisk.py`.

1. Вопросы к условию

1. _____
2. _____
3. _____
4. _____
5. _____

2. Карта вариантов

Вариант	Время	Доп. память	Что требует от входа
v01_lineyno_kazhdyy			
v02_operator_in			
v03_set_zarantee			
v04_sort_kopii_bisect			
v05_sort_na_meste_bisect			
v06_flagi			
v07_slovar_kolichestv			
v08_set_kazhdyy_raz			
v09_binarnyy_poisk_svoy			
v10_sliyanie_zaprosov			

3. Заказы

№	Ограничения заказа	Вариант	Почему он
B1	Один запрос к списку из 100 000 чисел, порядок произвольный		
B2	200 000 запросов к списку из 1 000 000 чисел, памяти достаточно		
B3	Список уже отсортирован по условию, 50 000 запросов, лишней памяти нет		

№	Ограничения заказа	Вариант	Почему он
B4	Значения — номера аудиторий от 0 до 999, запросов много		
B5	Нужно знать не только «есть ли», но и сколько раз встречается		

4. Варианты, которые не подойдут ни одному заказу

Проверка

Команда: `python3 -m unittest -v test_variantov.py`

Итог проверки (ОК или FAILED и сколько): _____

Замер (по желанию): `python3 zamer.py`. Какие варианты сравнили и что получилось по росту времени:

Сверка в паре

Участник 1 делает задачу А, участник 2 — задачу В, затем меняются. Расхождения в выборе и чем они объяснились:

Листинг: десять вариантов задачи А

Файл zadacha_a_kth.py из архива. Код на бумаге совпадает с файлом в архиве.

```
"""Задача А: вернуть k-й по величине элемент списка целых чисел.

Десять рабочих реализаций одной задачи. Они отличаются временем,
дополнительной памятью и требованиями к входу. Какая подходит,
определяют ограничения задачи, а их узнают вопросами к условию.

Общий контракт: k считается с единицы, повторы учитываются,
1 <= k <= len(numbers), исходный список не изменяется.
Варианты, которые нарушают этот контракт, отмечены в TREBOVANIYA.

Запуск: python3 zadacha_a_kth.py
"""

import heapq
import random
from bisect import insort

def v01_sort_kopii(numbers, k):
    """Сортировка копии по убыванию. O(n log n) времени, O(n) памяти."""
    return sorted(numbers, reverse=True)[k - 1]

def v02_sort_na_meste(numbers, k):
    """Сортировка на месте. O(n log n) времени, O(1) доп. памяти.
    Меняет исходный список: после вызова порядок элементов другой."""
    numbers.sort()
    return numbers[-k]

def v03_nlargest(numbers, k):
    """heapq.nlargest: k наибольших. O(n log k) времени, O(k) памяти."""
    return heapq.nlargest(k, numbers)[-1]

def v04_kucha_iz_k(numbers, k):
    """Своя куча из k наибольших. O(n log k) времени, O(k) памяти.
    Подходит, когда весь список сразу в память не помещается."""
    heap = []
    for value in numbers:
        if len(heap) < k:
            heapq.heappush(heap, value)
        elif value > heap[0]:
            heapq.heapreplace(heap, value)
    return heap[0]

def v05_k_prohodov(numbers, k):
    """k проходов: каждый раз ищем максимум среди неотброшенных.
    O(n * k) времени, O(n) памяти на копию. Для k = 1 это обычный поиск максимума."""
    ostatok = list(numbers)
    for _ in range(k - 1):
        ostatok.remove(max(ostatok))
    return max(ostatok)
```

```

def v06_quickselect(numbers, k):
    """Быстрый выбор. В среднем  $O(n)$ , в худшем случае  $O(n^2)$ .
    Работает с копией, исходный список не меняет."""
    chast = list(numbers)
    cel = len(chast) - k
    levo, pravo = 0, len(chast) - 1
    while True:
        opora = chast[random.randint(levo, pravo)]
        menshe = [x for x in chast[levo:pravo + 1] if x < opora]
        ravno = [x for x in chast[levo:pravo + 1] if x == opora]
        bolshe = [x for x in chast[levo:pravo + 1] if x > opora]
        if cel - levo < len(menshe):
            chast[levo:pravo + 1] = menshe
            pravo = levo + len(menshe) - 1
        elif cel - levo < len(menshe) + len(ravno):
            return opora
        else:
            levo = levo + len(menshe) + len(ravno)
            chast[levo:pravo + 1] = bolshe

def v07_schetchnik(numbers, k):
    """Счётчик по значениям.  $O(n + m)$  времени и  $O(m)$  памяти,
    где  $m$  — размер диапазона значений. Требуется целых значений от 0 до  $m$ ."""
    predel = max(numbers)
    schet = [0] * (predel + 1)
    for value in numbers:
        schet[value] += 1
    ostalos = k
    for value in range(predel, -1, -1):
        ostalos -= schet[value]
        if ostalos <= 0:
            return value

def v08_bisect_top_k(numbers, k):
    """Держим отсортированный список из  $k$  наибольших через insert.
     $O(n * k)$  в худшем случае из-за сдвигов при вставке,  $O(k)$  памяти."""
    top = []
    for value in numbers:
        insert(top, value)
        if len(top) > k:
            top.pop(0)
    return top[0]

def v09_dve_peremennye(numbers, k):
    """Один проход с двумя переменными.  $O(n)$  времени,  $O(1)$  памяти.
    Работает только при  $k = 2$ : для другого  $k$  нужен другой код."""
    if k != 2:
        raise ValueError('вариант v09 написан только для  $k = 2$ ')
    pervyy = vtoroy = None
    for value in numbers:
        if pervyy is None or value > pervyy:
            pervyy, vtoroy = value, pervyy
        elif vtoroy is None or value > vtoroy:
            vtoroy = value
    return vtoroy

def v10_razlichnye(numbers, k):
    """ $k$ -е различное значение: повторы сначала убираются.

```

Другой контракт: для [10, 10, 5] и k = 2 ответ 5, а не 10."

```
return sorted(set(numbers), reverse=True)[k - 1]
```

Что каждый вариант требует от входа и чем платит.

```
TREBOVANIYA = {
    'v01_sort_kopii': 'ничего не требует; O(n log n) времени, O(n) памяти',
    'v02_sort_na_meste': 'меняет исходный список; O(n log n) времени, O(1) доп. памяти',
    'v03_nlargest': 'ничего не требует; O(n log k) времени, O(k) памяти',
    'v04_kucha_iz_k': 'ничего не требует; O(n log k) времени, O(k) памяти',
    'v05_k_prohodov': 'ничего не требует; O(n * k) времени, O(n) памяти',
    'v06_quickselect': 'ничего не требует; в среднем O(n), в худшем O(n^2)',
    'v07_schetchik': 'целые значения от 0 до m; O(n + m) времени, O(m) памяти',
    'v08_bisect_top_k': 'ничего не требует; O(n * k) в худшем случае, O(k) памяти',
    'v09_dve_peremennnye': 'только k = 2; O(n) времени, O(1) памяти',
    'v10_razlichnye': 'другой контракт: k-е различное значение',
}
```

```
VARIANTY = [v01_sort_kopii, v02_sort_na_meste, v03_nlargest, v04_kucha_iz_k,
             v05_k_prohodov, v06_quickselect, v07_schetchik, v08_bisect_top_k,
             v09_dve_peremennnye, v10_razlichnye]
```

```
if __name__ == '__main__':
    dannye = [7, 2, 9, 9, 4, 1]
    k = 2
    print('вход:', dannye, '| k =', k)
    for variant in VARIANTY:
        try:
            print(f'{variant.__name__:20} {variant(list(dannye), k)}')
        except ValueError as oshibka:
            print(f'{variant.__name__:20} ошибка: {oshibka}')
```

Листинг: десять вариантов задачи В

Файл zadacha_b_poisk.py из архива. Код на бумаге совпадает с файлом в архиве.

```
"""Задача В: ответить на запросы «есть ли значение x в списке».
```

Десять рабочих реализаций одной задачи. Функция получает список чисел и список запросов, возвращает список ответов True/False в том же порядке. Отличаются они временем, дополнительной памятью и требованиями к входу.

Общий контракт: исходный список не изменяется, порядок ответов совпадает с порядком запросов. Варианты, которые нарушают этот контракт или требуют чего-то от входа, отмечены в TREBOVANIYA.

Запуск: `python3 zadacha_b_poisk.py`

```
"""
from bisect import bisect_left
from collections import Counter

def v01_lineyno_kazhdyy(numbers, zaprosy):
    """Для каждого запроса свой проход по списку. O(n * q) времени, O(1) памяти."""
    otvety = []
    for x in zaprosy:
        nayden = False
        for value in numbers:
            if value == x:
                nayden = True
                break
        otvety.append(nayden)
    return otvety

def v02_operator_in(numbers, zaprosy):
    """То же самое через оператор in. Короче на вид, но работа та же:
    in по списку — это линейный проход, всего O(n * q)."""
    return [x in numbers for x in zaprosy]

def v03_set_zaranee(numbers, zaprosy):
    """Множество строится один раз до запросов.
    O(n + q) времени в среднем, O(n) памяти."""
    vidennye = set(numbers)
    return [x in vidennye for x in zaprosy]

def v04_sort_kopii_bisect(numbers, zaprosy):
    """Отсортированная копия и двоичный поиск по ней.
    O(n log n + q log n) времени, O(n) памяти. Исходный список цел."""
    poryadok = sorted(numbers)
    otvety = []
    for x in zaprosy:
        mesto = bisect_left(poryadok, x)
        otvety.append(mesto < len(poryadok) and poryadok[mesto] == x)
    return otvety

def v05_sort_na_meste_bisect(numbers, zaprosy):
    """Сортировка на месте и двоичный поиск. O(n log n + q log n) времени,
```

```

O(1) доп. памяти. Меняет исходный список. """
numbers.sort()
otvety = []
for x in zaprosy:
    mesto = bisect_left(numbers, x)
    otvety.append(mesto < len(numbers) and numbers[mesto] == x)
return otvety

def v06_flagi(numbers, zaprosy):
    """Массив флагов по значениям. O(n + m + q) времени, O(m) памяти,
    где m — размер диапазона. Требуется целых значений от 0 до m. """
    predel = max(numbers)
    est = [False] * (predel + 1)
    for value in numbers:
        est[value] = True
    return [0 <= x <= predel and est[x] for x in zaprosy]

def v07_slovar_kolichestv(numbers, zaprosy):
    """Словарь «значение → сколько раз». O(n + q) времени, O(n) памяти.
    Дает не только ответ «есть ли», но и кратность. """
    skolko = Counter(numbers)
    return [skolko[x] > 0 for x in zaprosy]

def v08_set_kazhdyy_raz(numbers, zaprosy):
    """Множество строится внутри каждого запроса. Проверка быстрая,
    но построение стоит O(n), поэтому всего снова O(n * q). """
    return [x in set(numbers) for x in zaprosy]

def v09_binarnyy_poisk_svoy(numbers, zaprosy):
    """Свой двоичный поиск без сортировки. O(q log n) времени, O(1) памяти.
    Требуется, чтобы список уже был отсортирован по возрастанию. """
    otvety = []
    for x in zaprosy:
        levo, pravo = 0, len(numbers) - 1
        nayden = False
        while levo <= pravo:
            seredina = (levo + pravo) // 2
            if numbers[seredina] == x:
                nayden = True
                break
            if numbers[seredina] < x:
                levo = seredina + 1
            else:
                pravo = seredina - 1
        otvety.append(nayden)
    return otvety

def v10_sliyanie_zaprosov(numbers, zaprosy):
    """Сортируем и список, и запросы, идём по обоим один раз.
    O(n log n + q log q) времени, O(n + q) памяти. Порядок ответов
    восстанавливается по исходным позициям запросов. """
    poryadok = sorted(numbers)
    pary = sorted(range(len(zaprosy)), key=lambda i: zaprosy[i])
    otvety = [False] * len(zaprosy)
    i = 0
    for nomer in pary:
        x = zaprosy[nomer]

```

```

while i < len(poryadok) and poryadok[i] < x:
    i += 1
otvety[nomer] = i < len(poryadok) and poryadok[i] == x
return otvety

```

```

TREBOVANIYA = {
    'v01_lineyno_kazhdyy': 'ничего не требует;  $O(n * q)$  времени,  $O(1)$  памяти',
    'v02_operator_in': 'ничего не требует;  $O(n * q)$  времени,  $O(1)$  памяти',
    'v03_set_zarantee': 'ничего не требует;  $O(n + q)$  времени,  $O(n)$  памяти',
    'v04_sort_kopii_bisect': 'ничего не требует;  $O(n \log n + q \log n)$ ,  $O(n)$  памяти',
    'v05_sort_na_meste_bisect': 'меняет исходный список;  $O(n \log n + q \log n)$ ,  $O(1)$  доп. памяти',
    'v06_flagi': 'целые значения от 0 до m;  $O(n + m + q)$  времени,  $O(m)$  памяти',
    'v07_slovar_kolichestv': 'ничего не требует;  $O(n + q)$  времени,  $O(n)$  памяти, даёт кратность',
    'v08_set_kazhdyy_raz': 'ничего не требует;  $O(n * q)$  времени: множество строится заново на каждый запрос',
    'v09_binarnyy_poisk_svoy': 'список уже отсортирован;  $O(q \log n)$  времени,  $O(1)$  памяти',
    'v10_sliyanie_zaprosov': 'ничего не требует;  $O(n \log n + q \log q)$ ,  $O(n + q)$  памяти',
}

```

```

VARIANTY = [v01_lineyno_kazhdyy, v02_operator_in, v03_set_zarantee,
             v04_sort_kopii_bisect, v05_sort_na_meste_bisect, v06_flagi,
             v07_slovar_kolichestv, v08_set_kazhdyy_raz, v09_binarnyy_poisk_svoy,
             v10_sliyanie_zaprosov]

```

```

if __name__ == '__main__':
    dannye = [4, 1, 9, 4, 7]
    zaprosy = [4, 5, 9, 0]
    print('список:', dannye, '| запросы:', zaprosy)
    for variant in VARIANTY:
        rabochiy = sorted(dannye) if variant is v09_binarnyy_poisk_svoy else list(dannye)
        print(f'{variant.__name__:26} {variant(rabochiy, list(zaprosy))}')

```